

Relatório 12 - Prática: Projeto Final (V)

Mauro Andrade

Descrição da atividade

O projeto LitterWatch foi desenvolvido como prova de conceito para detectar e contar visitas de um gato a uma caixa de areia. A proposta surgiu da dificuldade de acompanhar esse comportamento manualmente, principalmente durante a noite ou quando o responsável está ausente. O sistema não diagnostica doenças e não confirma se houve micção ou defecação; ele apenas demonstra como dados sintéticos e visão computacional podem apoiar o registro de presença, início, fim e duração de visitas.

A cena foi criada programaticamente no Blender 3.6.23. Para manter o projeto compatível com o computador disponível, foi utilizado um gato 3D simplificado formado por treze componentes geométricos. Também foram modelados a base, as paredes e a areia da caixa. Um script em Python com a API bpy configurou câmera, piso, materiais, iluminação e renderização.



Figura 1 — Cena-base criada no Blender com gato e caixa de areia.

O gerador automatizou sessenta imagens sintéticas em resolução de 320 por 320 pixels. Foram criadas vinte cenas com o gato dentro da caixa, vinte próximo e vinte fora. Posição, rotação, cor do gato, intensidade e posição da luz e altura da câmera foram variadas. O dataset foi dividido em 48 imagens de treino, 6 de validação e 6 de teste, com semente fixa para tornar o experimento reproduzível.

As anotações foram geradas sem marcação manual. Os vértices das caixas tridimensionais de todas as partes do gato foram projetados na câmera e reunidos em uma única bounding box no formato YOLO. Um segundo programa verificou a presença de sessenta pares de imagem e rótulo, valores normalizados entre zero e um e caixas com área positiva.

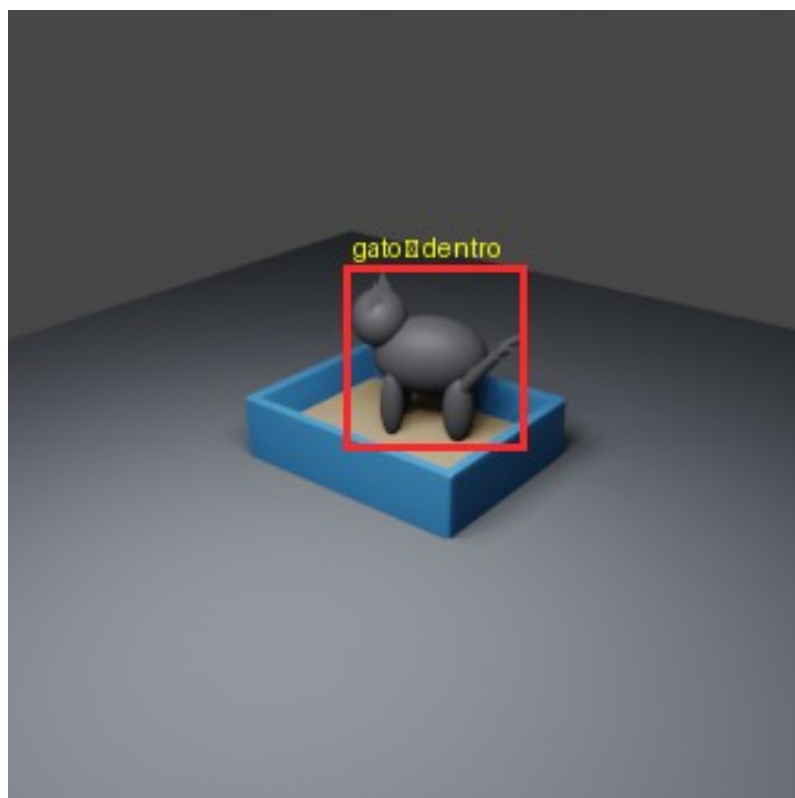


Figura 2 — Exemplo de anotação automática da classe gato.

Foi utilizado o modelo YOLO26n por transferência de aprendizado. A arquitetura nano foi escolhida para reduzir o custo computacional. O treinamento foi realizado em CPU por oito épocas, com imagens de 320 pixels, batch 2 e uma única classe, denominada gato. O objetivo foi validar o pipeline completo e não produzir um sistema clínico ou comercial.

Na validação sintética, o modelo alcançou mAP50 de 0,942. No conjunto de teste separado, o mAP50 foi 0,636. A diferença entre os valores indica que o conjunto pequeno não permite afirmar forte capacidade de generalização. Ainda assim, as seis imagens da demonstração receberam detecções, permitindo comprovar o funcionamento técnico do fluxo.

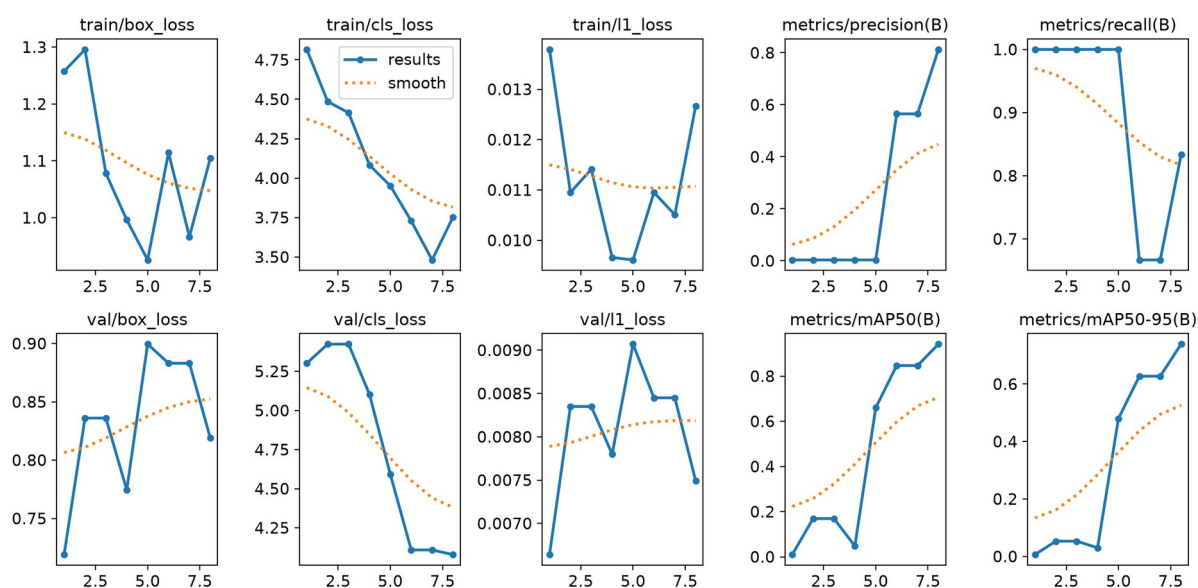


Figura 3 — Gráficos produzidos durante as oito épocas de treinamento.

Por fim, foi implementada uma máquina de estados para transformar quadros consecutivos em visitas. O contador exige duas confirmações para entrada e duas para saída, evitando que uma detecção isolada seja interpretada como nova visita. Em uma sequência simulada, foram contabilizadas duas visitas, com durações de cinco e quatro segundos, e um ruído isolado foi ignorado.

Dificuldades

A primeira dificuldade foi a limitação de hardware. A versão mais recente do Blender apresentou incompatibilidade gráfica, por isso foi adotado o Blender 3.6.23 e o Eevee com renderização em CPU. O treinamento também foi configurado com modelo nano, resolução pequena e batch reduzido.

Outra dificuldade foi representar um gato sem utilizar um ativo 3D pesado. A solução foi construir um modelo simplificado com formas geométricas, suficiente para demonstrar scripting, randomização e anotação. Durante os testes, a contagem esperada das partes estava incorreta; o teste automático revelou que o modelo possuía treze componentes e a validação foi corrigida antes da geração do dataset.

Também foi necessário calcular uma única caixa para um objeto composto. A solução consistiu em projetar todos os vértices das treze partes na imagem e combinar seus limites. Depois, previews com retângulos desenhados foram inspecionados visualmente para confirmar o alinhamento.

A principal dificuldade conceitual foi manter o escopo responsável. Contar visitas pode auxiliar a observação, mas não identifica micção, volume urinário ou doença. Por isso, diagnóstico, sensores, câmera ao vivo e múltiplos gatos foram explicitamente retirados do MVP.

Conclusões

O trabalho integrou as etapas exigidas no desafio: modelagem e scripting no Blender, randomização, geração de imagens, anotação automática, organização do dataset, treinamento, avaliação e demonstração de uma regra de negócio. O uso de scripts tornou o processo reproduzível e eliminou a necessidade de anotar manualmente cada imagem.

Os resultados sintéticos confirmam a viabilidade acadêmica, mas não comprovam funcionamento no mundo real. O gato simplificado, o conjunto pequeno e a ausência de imagens reais criam um domain gap significativo. Uma continuação exigiria dados reais autorizados, avaliação em vídeo, sensores complementares e participação de profissionais veterinários.

Concluo que dados sintéticos são úteis para construir e testar um pipeline quando dados reais são escassos, mas suas métricas devem ser interpretadas com cuidado. O projeto cumpre seu objetivo como prova de conceito educacional e mantém transparentes suas limitações.

Referências

BLENDER FOUNDATION. Blender Python API Documentation. Disponível em: <https://docs.blender.org/api/3.6/>.

ULTRALYTICS. Ultralytics YOLO Documentation. Disponível em: <https://docs.ultralytics.com/>.

DENG, P. et al. Quantification of Urine Elimination Behaviors in Cats with a Video Recording System. Journal of Veterinary Internal Medicine, 2017.

SUNG, Y. et al. Multi-Cat Monitoring System Based on Concept Drift Adaptive Machine Learning Architecture. Sensors, 2023.

EMC/UFG. FastCamp de Dados Sintéticos para Inteligência Artificial e Visão Computacional, 2026.

Código e artefatos do projeto: <https://github.com/mhaurinho/litterwatch-dados-sinteticos>.